

Introduction to R and Data Frames

GOVT 10: Quantitative Political Analysis

This chapter introduces R and the data frame, the workhorse structure for political data. You will learn to store values in objects, work with vectors, load data with the tidyverse, and examine a dataset before analyzing it.

1 Introduction to R and Data Frames

The second half of the chapter shifts from why we use data to how. Quantitative analysis is impossible without software, and the software we will use throughout this course is R.

1.1 What R Is and Why We Use It

R is a programming language designed for statistical computing and data analysis. It is free, open source, widely used in academia and industry, and supported by an enormous community of contributors. When a political scientist publishes an analysis in a journal, the odds are very good that the underlying code is written in R. When a polling firm builds an election model, R is often part of the pipeline.

R has two qualities that make it ideal for this course. First, it is honest: every step of your analysis is written down in code, so you (and your future self, and anyone checking your work) can see exactly what you did. Second, it is general: the same language that handles a small classroom dataset can handle the millions of rows in a national voter file.

1.2 The Role of Positron

We will not interact with R directly through a command line. We will use **Positron**, a modern integrated development environment (IDE) designed for data science. Positron gives you a code editor, a console for running R, a place to see your data, and a place to view plots, all in one window. It also makes it easy to work with AI assistants alongside your code.

For now, you only need to know two things. First, you write R code inside Positron in a `.qmd` file (a Quarto document). Second, you run that code chunk by chunk, watching the output appear in the console below. We will walk through the mechanics in class.

A Note on Vocabulary

Students sometimes confuse "R" and "Positron." R is the *language*; it is what the computer

actually runs. Positron is the *interface* where you write and run R code. You could write R in many different editors. We use Positron because it makes the workflow smooth.

1.3 Storing Information: Variables and Assignment

The most basic thing R does is store information in **objects**. You create an object using the assignment operator `<-`. Think of an object as a labeled container that holds a value.

```
house_seats_to_majority <- 218
house_seats_to_majority
```

```
[1] 218
```

```
senator_name <- "Warnock"
senator_name
```

```
[1] "Warnock"
```

```
turnout_rate_2022 <- 107710427 / 240452185
turnout_rate_2022
```

```
[1] 0.4479495
```

What this code does: We store three values. `house_seats_to_majority` holds the number 218 (the threshold for a majority in the U.S. House). `senator_name` holds the text "Warnock." `turnout_rate_2022` holds the result of a calculation: total ballots cast in the 2022 midterm divided by the citizen voting-age population. Typing the variable name alone tells R to print its contents.

Two small conventions matter. First, R uses `<-` (an arrow) rather than `=` for assignment. The arrow makes it visually clear which direction the value is going. Second, variable names cannot contain spaces. Use underscores between words: `house_seats_to_majority`, not `house seats to majority`.

1.4 Variable Types

R automatically figures out what kind of data you have given it. The three types you will use most often are:

- **Numeric:** numbers used in calculations, like 218 or 47.3 or -5.2. Used for vote counts, percentages, ages, dollar amounts.
- **Character:** text inside quotes, like "Warnock" or "Democratic". Used for names, party labels, state abbreviations, open-ended survey responses.
- **Logical:** the values TRUE and FALSE. Used for yes/no conditions, such as whether a state was carried by the incumbent.

The data type matters because it determines what operations are legal. You can take the mean of a vector of numbers; you cannot take the mean of a vector of state names.

```
number_value <- 218
text_value <- "218"

class(number_value)
```

```
[1] "numeric"
```

```
class(text_value)
```

```
[1] "character"
```

What this code does: Both `number_value` and `text_value` display as 218, but they are different. `class()` reveals that one is "numeric" and the other is "character." If you tried to do math with the character version, R would refuse. This is a common source of confusion: data that looks numeric can be stored as text, and you have to check.

1.5 Vectors: Collections of Values

We often want to store more than one value under a single name. The turnout rates of G7 democracies, the approval ratings of every president since FDR, the populations of every country in the UN. A **vector** is an ordered collection of values of the same type, and the function `c()` (for "combine") creates one.

```
g7_countries <- c("United States", "United Kingdom", "France",
                  "Germany", "Italy", "Canada", "Japan")
turnout_pct <- c(66.6, 67.3, 71.6, 76.6, 63.9, 62.6, 55.9)
```

```
g7_countries
```

```
[1] "United States" "United Kingdom" "France"          "Germany"         "Italy"
[6] "Canada"        "Japan"
```

```
turnout_pct
```

```
[1] 66.6 67.3 71.6 76.6 63.9 62.6 55.9
```

What this code does: We create two vectors. `g7_countries` stores the names of the seven G7 democracies. `turnout_pct` stores the voter turnout rate (as a percentage) in each country's most recent national legislative election. Order matters: the first element of one vector goes with the first element of the other.

All elements of a vector must be the same type. If you try to mix types, R silently converts everything to a common type, usually character. This is called coercion and it can quietly break your analysis if you do not notice.

```
mixed <- c(1, 2, "three", 4)
mixed
```

```
[1] "1"      "2"      "three"  "4"
```

```
class(mixed)
```

```
[1] "character"
```

What this code does: Because the value "three" cannot be a number, R converts the numbers 1, 2, and 4 to text. Notice the quotes around them in the output. The vector is now of class "character," which means `mean(mixed)` will fail.

1.6 Basic Vector Operations

Once you have a vector, R makes it easy to ask questions about it. A few essentials:

```
length(turnout_pct)
```

```
[1] 7
```

```
sum(turnout_pct)
```

```
[1] 464.5
```

```
max(turnout_pct)
```

```
[1] 76.6
```

```
min(turnout_pct)
```

```
[1] 55.9
```

What this code does: `length()` counts how many elements are in the vector (here, seven G7 countries). `sum()` adds them up (the total has no substantive meaning across countries, but it shows the function at work). `max()` and `min()` report the largest and smallest values, in this case Germany's high and Japan's low. These four functions, applied to a vector of numbers, already give you a basic summary of the data.

R also lets you do math on an entire vector at once. If you wanted to express each turnout rate as a proportion (between 0 and 1) rather than a percentage, you can divide the whole vector by 100 in a single line:

```
turnout_pct / 100
```

```
[1] 0.666 0.673 0.716 0.766 0.639 0.626 0.559
```

You did not have to write a loop. R applied the division to every element automatically. This is called **vectorized** computation, and it is one of the reasons R is so well suited to data analysis.

1.7 What a Data Frame Is

A vector is one column of data. Most political datasets have many columns: a row for each respondent or country or election, and columns for every variable measured. The structure that holds this kind of rectangular data is called a **data frame**.

You can think of a data frame as a spreadsheet inside R. Rows are observations (one survey respondent, one country, one election). Columns are variables (party identification, GDP, vote share). Every column has a name. Every column in a given data frame has the same number of rows.

Why Data Frames Matter

Almost everything we do in this course will start with a data frame. When you load a CSV file from disk, you get a data frame. When you summarize survey results, you get a data frame. When you fit a regression model and ask for the coefficients in a tidy format, you get a data frame. Becoming fluent with data frames is the single most important skill of the first few weeks.

1.8 Loading Real Data with `read_csv()`

In this course, the data we work with will almost always live in a CSV (comma-separated values) file. A CSV is just a plain text file where each row is one observation and the columns are separated by commas. The first row is usually the column names.

To load a CSV into R, we use the `read_csv()` function from the **tidyverse**. The tidyverse is a collection of R packages designed to work together using a consistent grammar. We have to load it at the start of every session:

```
library(tidyverse)
approval <- read_csv("../data/presidential_approval.csv")
```

What this code does: `library(tidyverse)` loads the tidyverse package and makes its functions available. `read_csv()` reads the file at the given path and returns a data frame, which we assign to the name `approval`. From now on, `approval` is an object we can poke at. The `presidential_approval.csv` file contains monthly Gallup approval ratings for the sitting U.S. president since 1980.

A Common First-Day Error

If you forget to run `library(tidyverse)`, you will get a message like `could not find function "read_csv"`. This is not a bug in R or in your data. R simply does not know what `read_csv()` is until you load the package that defines it. Always load your libraries first.

1.9 Examining a Data Frame

Once data is loaded, the first thing to do is look at it. R provides a small set of functions that should be the first thing you reach for whenever you meet a new dataset.

The most useful is `glimpse()`. It shows you the dimensions of the data frame, the name and type of every column, and the first few values in each column, all in a compact display.

```
glimpse(approval)
```

```
Rows: 528
Columns: 2
$ date      <date> 1980-01-15, 1980-02-15, 1980-03-15, 1980-04-15, 1980-05-15, 198~
```

```
$ approval_rating <dbl> 58.2, 55.8, 39.4, 40.1, 43.2, 31.8, 33.1, 32.8, 37.5, 37.0, 34.4~
```

What this code does: `glimpse()` tells us how many rows and columns `approval` has, and the type of each column. The `date` column is a `<date>`, and `approval_rating` is `<dbl>` (a numeric type). We can see the first values in each column on the right.

When you see a `<chr>` next to a column, that column is text. `<dbl>` means a decimal number, `<int>` means a whole-number integer, `<lgl>` means logical, and `<date>` means a date. These labels are important because they tell you what operations are legal on each column.

The `head()` function shows the first six rows of the actual data:

```
head(approval)
```

```
# A tibble: 6 x 2
  date      approval_rating
  <date>      <dbl>
1 1980-01-15          58.2
2 1980-02-15          55.8
3 1980-03-15          39.4
4 1980-04-15          40.1
5 1980-05-15          43.2
6 1980-06-15          31.8
```

What this code does: `head()` prints the top of the data frame so you can see real values, not just structure. There is a companion function `tail()` that shows the last few rows, useful when you want to check whether your data extends to the most recent year you expected.

The `summary()` function gives a quick statistical summary of each column. For numeric columns it reports the minimum, first quartile, median, mean, third quartile, and maximum. For character columns it reports counts.

```
summary(approval)
```

	date	approval_rating
Min.	:1980-01-15	Min. :23.00
1st Qu.	:1991-01-07	1st Qu.:42.30
Median	:2001-12-30	Median :48.00
Mean	:2001-12-29	Mean :50.42
3rd Qu.	:2012-12-22	3rd Qu.:58.00
Max.	:2023-12-15	Max. :87.00

To compute a single number, like the mean of one column, refer to that column with the dollar sign:

```
mean(approval$approval_rating)
```

```
[1] 50.41989
```

What this code does: `approval$approval_rating` pulls out the `approval_rating` column as a vector. `mean()` then averages every monthly observation to give the average presidential approval rating over the entire period covered by the dataset.

For categorical data, `table()` counts how many observations fall into each category. Suppose we have a small vector of congressional party identifications:

```
party <- c("D", "R", "R", "D", "I", "D", "R", "R", "D", "D")
table(party)
```

```
party
D I R
5 1 4
```

What this code does: `table()` counts how many times each unique value appears: five Democrats, four Republicans, one Independent. This is the basic tool for summarizing nominal (categorical) variables.

1.10 Tying the Two Segments Together

Why did we put a discussion of John Snow and a tour of R in the same chapter? Because they are the same idea expressed two different ways. Snow had a hypothesis, collected data systematically, looked for patterns, and acted on the evidence. We are going to do the same thing all term, except our hypotheses will be about turnout, persuasion, polarization, and policy, and our tool for collecting patterns will be R rather than a hand-drawn map.

Every week, the loop will look the same. We will start with a question we care about politically. We will find or build a dataset that bears on the question. We will load it into R, look at its structure with `glimpse()`, examine its values with `head()` and `summary()`, summarize it with `mean()` or `table()`, and use that summary to say something honest about politics.

Looking Ahead

In the next chapter we will move from describing single variables to manipulating real political datasets: filtering them to the rows we care about, selecting the columns we need, sorting them, and computing summary statistics by group. The R functions you met this week (especially `library()`, `read_csv()`, `glimpse()`, and `mean()`) are the foundation. Get comfortable typing them.

2 Using AI for This Material

Where AI Helps This Week

- Composing a dplyr pipeline from a verbal description, especially long chains of `filter()` into `group_by()` into `summarise()`.
- Explaining the differences between `filter()`, `select()`, and `slice()` and when each is the right verb.
- Sketching what a `group_by()` `%>% summarise()` chain will return before you run it.

Where AI Gets This Wrong This Week

- Choosing the mean versus the median for a specific variable requires looking at the distribution, which AI cannot do.
- Deciding whether to drop NAs or impute them depends on why the data are missing in your particular dataset, which AI cannot diagnose.
- It will sometimes assert the 68-95-99.7 rule applies when your variable is skewed, bounded, or discrete and the rule does not actually hold.

A prompt that works for this week:

Write a dplyr pipeline that takes a data frame called `polls`, filters to 2024 polls in swing states, groups by `state`, and returns the mean and median `dem_share` per state, ordered by mean descending.

A prompt that misleads:

Should I report the mean or the median for income in my dataset?

Why it misleads: choosing between mean and median requires inspecting the distribution's skew and outliers, which AI cannot do without seeing your data — it will default to a generic “use median for skewed data” without knowing whether yours is.

3 Common Mistakes and How to Avoid Them

Mistake 1: Forgetting `na.rm = TRUE`

If any value in a vector is NA, then `mean(x)`, `median(x)`, and `sd(x)` all return NA. This is R protecting you, not failing. Add `na.rm = TRUE` when you intend to ignore missingness, but only after you have checked how much data is actually missing.

Mistake 2: Reporting the Mean for Skewed Data

When campaign contributions are reported as means, the picture is misleading. A handful of large donors pulls the mean far above what the typical donor gives. Always visualize the distribution first. If the histogram has a long tail, report the median, or report both and explain why they differ.

Mistake 3: Treating Correlation as Causation

This cannot be emphasized enough. Two variables can move together because one causes the other, because the other causes the one, because both are caused by something else, or by pure chance. Correlation alone cannot distinguish among these. Careful research design — the subject of Week 4 — is what lets us make causal claims.

Mistake 4: Using `==` to Check for Missing Values

The expression `x == NA` does not return `TRUE` when `x` is missing; it returns `NA`. Use `is.na(x)` to identify missing values and `!is.na(x)` to identify present ones.

4 R Functions Reference

Function	Purpose	Example
<code>names(df)</code>	List column names	<code>names(states)</code>
<code>filter()</code>	Keep rows matching conditions	<code>filter(party == "D")</code>
<code>select()</code>	Keep specified columns	<code>select(name, vote)</code>
<code>arrange()</code>	Sort rows	<code>arrange(desc(votes))</code>
<code>is.na(x)</code>	Test for missing values	<code>is.na(income)</code>
<code>!is.na(x)</code>	Test for non-missing values	<code>filter(!is.na(income))</code>

Table 1: Working with Real Data (Segment A)

Function	Purpose	Example
<code>mean(x)</code>	Arithmetic mean	<code>mean(x, na.rm = TRUE)</code>
<code>median(x)</code>	Median	<code>median(income)</code>
<code>sd(x)</code>	Standard deviation	<code>sd(approval)</code>
<code>var(x)</code>	Variance	<code>var(scores)</code>
<code>n()</code>	Count rows within <code>summarise()</code>	<code>summarise(n = n())</code>
<code>group_by()</code>	Group data for calculations	<code>group_by(state)</code>
<code>summarise()</code>	Compute summary statistics	<code>summarise(m = mean(x))</code>
<code>ggplot()</code>	Initialize a plot	<code>ggplot(df, aes(x = v))</code>
<code>geom_histogram()</code>	Add a histogram layer	<code>geom_histogram(bins = 30)</code>
<code>geom_density()</code>	Add a density layer	<code>geom_density()</code>
<code>geom_vline()</code>	Add a vertical reference line	<code>geom_vline(xintercept = 50)</code>

Table 2: Summary Statistics and Visualization (Segment B)

5 Practice Problems

The following questions are designed to test your understanding of the material in this chapter. Work through them before class, and bring your questions to discussion.

Question 1

You load a dataset of 1,000 survey respondents and find that 220 of them have a missing value for the `income` variable. You decide to compute the mean income with `mean(income, na.rm = TRUE)`. What assumption are you making about the 220 missing cases? Under what circumstances would this assumption be problematic?

Question 2

Write a single tidyverse pipeline that takes the `states` data frame, keeps only Southern states from the 2020 election, sorts from highest to lowest Democratic vote share, and shows the first five rows of `state`, `region`, and `democratic_vote_share`. Label each step with a comment explaining what it does.

Question 3

A news article reports that "the average American household has \$1.06 million in net worth." A friend who reads this concludes that most Americans are doing well financially. Explain in two or three sentences why this conclusion is probably wrong, using the language of mean, median, and skewness.

Question 4

Researchers report that across U.S. states, the correlation between the number of Starbucks locations and average SAT scores is $r = 0.41$. Propose at least two non-causal explanations for this correlation. What would you need to do to test whether Starbucks locations actually cause SAT improvements?

Question 5 (Computational)

Using the `states` data, write a grouped summary that, for each combination of `region` and `year`, reports the number of states, the mean Republican vote share, and the standard deviation of the Republican vote share. Which region-year combinations show the most internal disagreement (largest SD), and which the most uniformity (smallest SD)? Use `arrange()` to find out.

For Further Study

This chapter has introduced the working pattern of modern political data analysis: load a real dataset, wrangle it with tidyverse verbs, handle missingness honestly, and summarize it with statistics that match the shape of the distribution. Students who want to go deeper might explore Hadley Wickham and Garrett Grolemund's *R for Data Science*, which is freely available online and covers tidyverse in depth. For a careful treatment of why correlation is not causation, Judea Pearl and Dana Mackenzie's *The Book of Why* is accessible and engaging. In the coming classes we will build on this foundation with summary statistics and by creating new variables with `mutate()`, and we will turn our attention to the question of when data can support causal claims.